

SUPSI

MineSweeper

Studente/i	Relatore	Correlatore
Xhyra Niccolò	Giancarlo Corti	Tommaso Agnola
Corso di laurea	Modulo / Codice Progetto	Anno
Ingegneria Informatica	Software Engineering I	2024/2025
Committente	Data	
Giancarlo Corti, Tommaso Agnola	27.08.2025	

Table of contents

- Context & Motivation
- Problem/Requirements
- Approach
- Demo
- Conclusions

Context

Desktop GUI application
development

Software Engineering
Principles



Motivation

Practice Java
Object-Oriented-Programming

Interactive UI

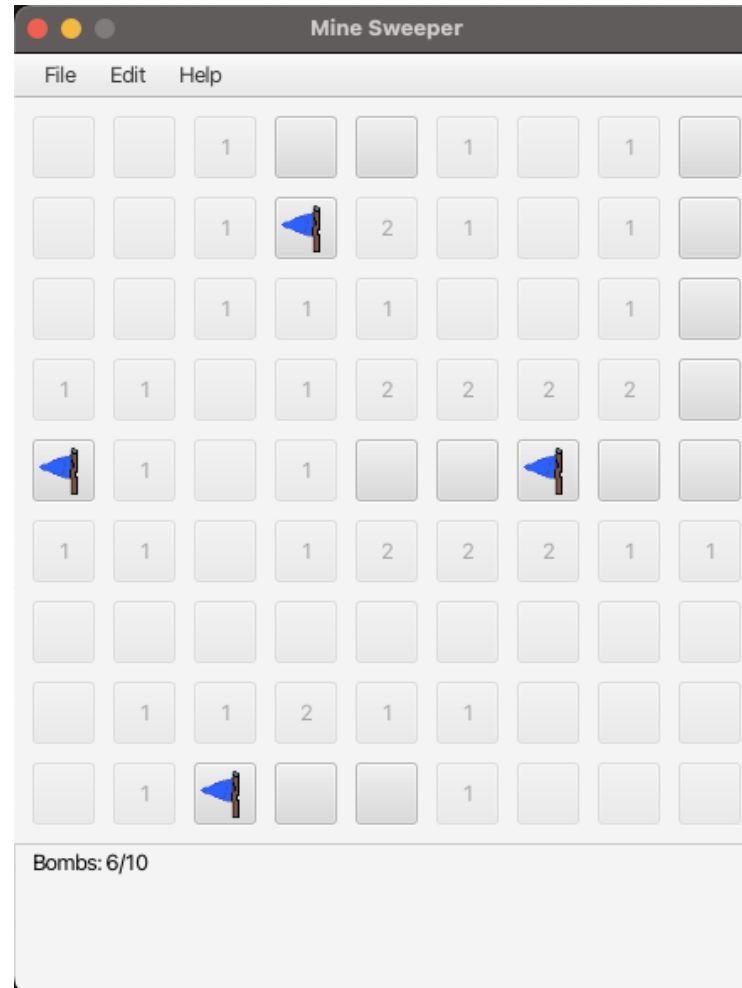


Problem

Develop MineSweeper

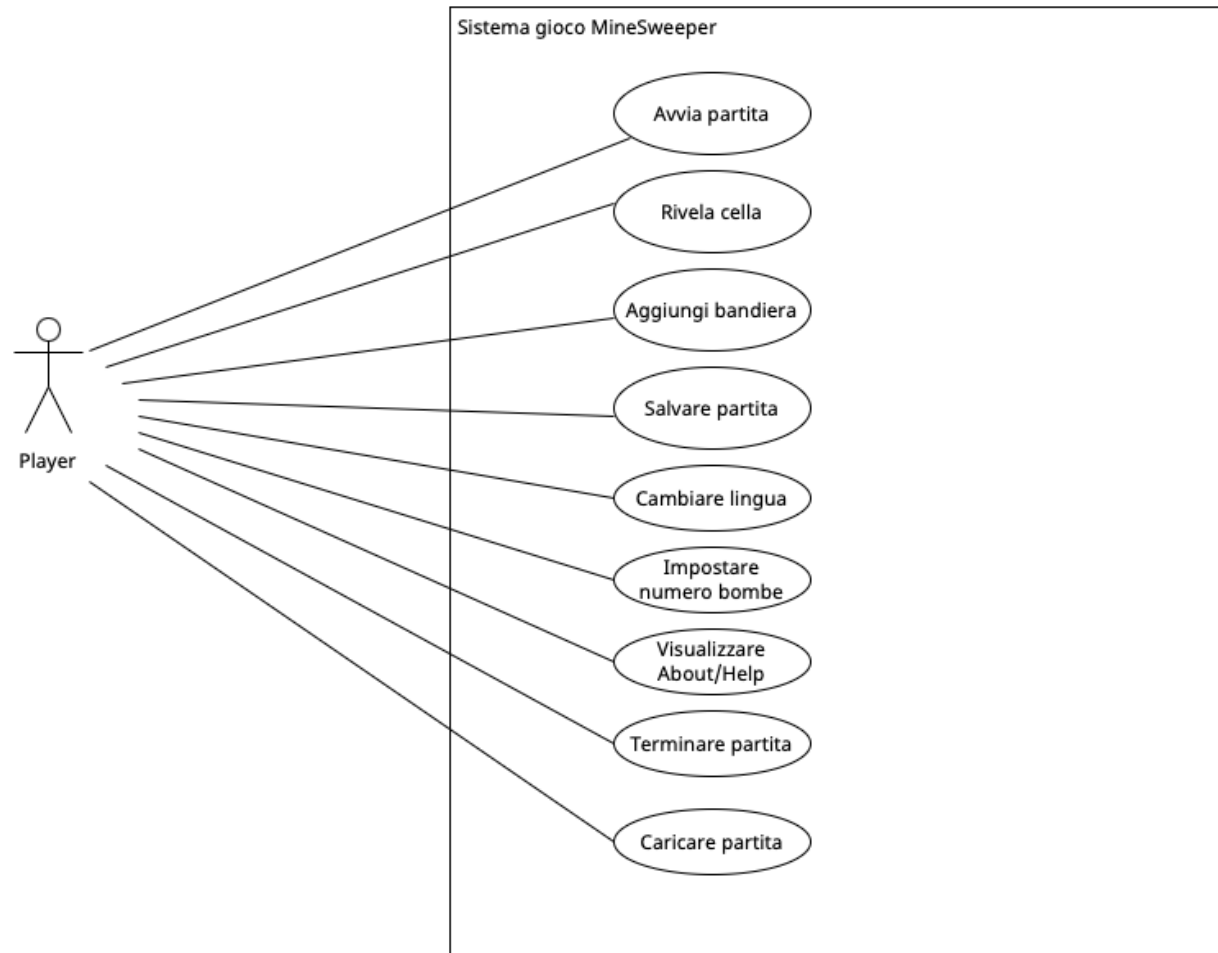
Allow Setting Customization

Persistence Saving-Loading



Requirements

- Interactive gameplay
- Menu and Cold reloaded Preferences
- Multi Module
- Save and Load feature
- Multiple Language Support



Approach

- **BE → Architettura a strati (Layered)**
- **FE → MVC:**

Approach – Backend

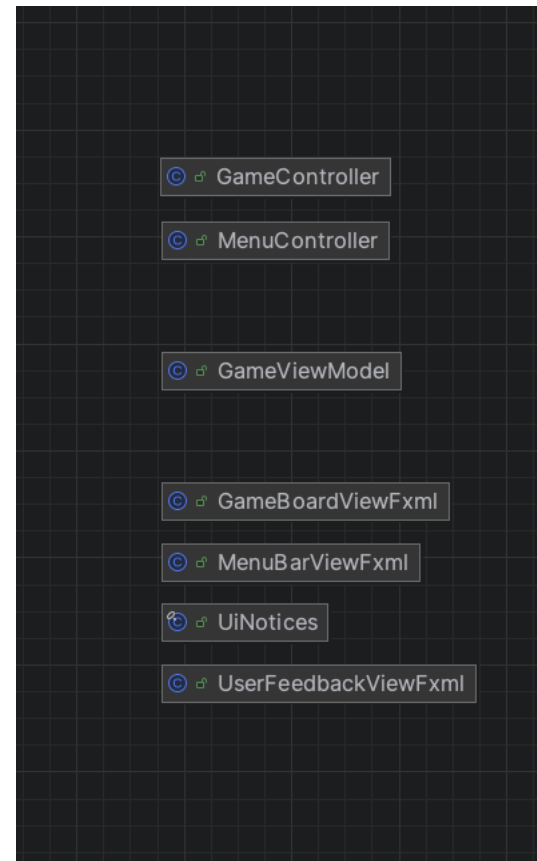
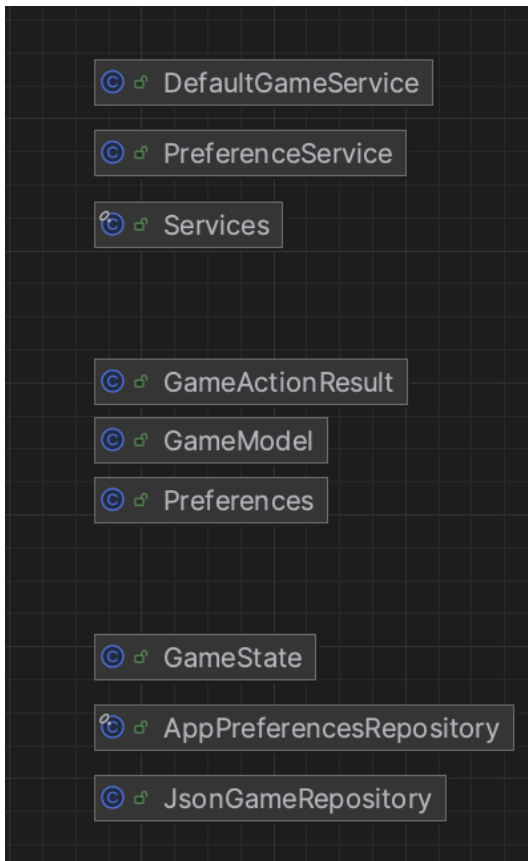
- **Model:** Stato della board, regole, invarianze (win/lose, flag, reveal).
- **Application (service):** Orchestrazione dei casi d'uso: `newGame(bombs)`, `handleClick(r,c,rc)`, `save/load`.
- **Repository (persistence):** I/O su disco (JSON per partita, `.properties` per preferenze) tramite `GameState`.

Approach – Frontend

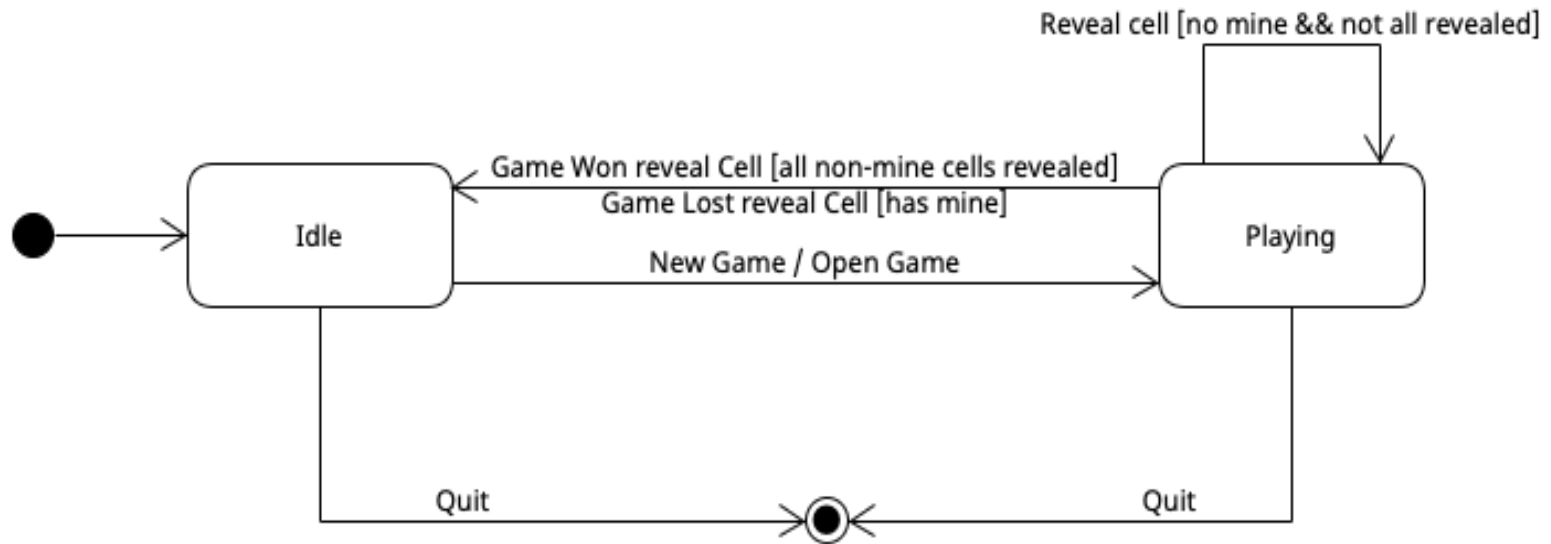
- **View (FXML + JavaFX):** La View disegna e inoltra eventi
- **Controller:** I Controller coordinano
- **Model:** Il ViewModel fa da adattatore verso il dominio/persistenza del BE.

View → Controller → Model (FE) → Service/Model (BE).

Approach



Approach



Demo

Conclusions

